

CC3001 Algoritmos y Estructuras de Datos**Profesores:** Nelson Baloian, Patricio Poblete, e Iván Sipirán**Auxiliares:** Sebastián Acuña, Daniela Espinoza, Cristián Llull Torres,

Benjamín Osses, Anish Samtani, y Marcelo Zamorano

**Auxiliar 5**

15 de mayo de 2026

P1. Encontrar el K-ésimo elemento en un ABB

Daniela y Aidan acaban de aprender cómo funciona un árbol de búsqueda binaria en su clase de Algoritmos y estructuras de datos. Daniela quiere desafiar a Aidan y comprobar que realmente aprendió cómo funciona este árbol, así que decide plantearle el siguiente problema:

Dados n enteros aleatorios en un arreglo, Daniela quiere saber cuál es el k -ésimo elemento del arreglo. El problema es que Aidan no prestó mucha atención en clases, por lo que te pide a ti ayuda para programarlo.

(a) Aidan necesita que programes una función `def getKthElement(arr, k)` que dado un arreglo, entregue el k -ésimo elemento.

Ahora que ayudaste a Aidan a lograr el desafío de Daniela, te enteras de que Daniela sigue sin creer que tu amigo Aidan sabe en verdad cómo ocupar un ABB. Es por esto que le plantea otro desafío:

(b) Dado un arreglo con n enteros aleatorios, debes programar una función `def getAllKthElements(arr, k)` que entregue todos los primeros k -ésimos elementos del arreglo.

P2. Move to Root en ABBs

Supongamos que se tiene un ABB ordenado como en la Figura 1, donde se fueron insertando los elementos en orden, de forma tal que el último insertado queda en las hojas del árbol.

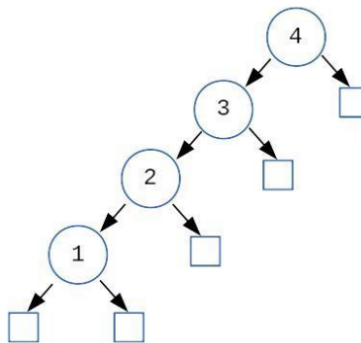


Figura 1: Árbol construido con peor caso.

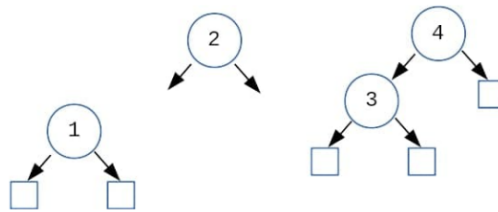


Figura 2: Corte vertical del árbol en nodo 2.

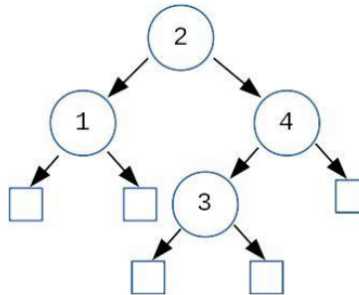


Figura 3: Resultado final del corte del árbol.

Suponiendo que la probabilidad de acceder al elemento más profundo es mayor que la de acceder al elemento de más arriba en el árbol, notamos que la construcción del árbol no es óptima. Luego, si repetimos la operación de acceso muchas veces, la solución no será óptima.

Para evitar este problema, existe una operación llamada “Move to Root” (muy parecida a *move to front*), que mueve el nodo por último accedido a la raíz del árbol. El algoritmo autoorganizante funciona de la siguiente manera: cuando se accede a un nodo, se corta el árbol verticalmente, obteniendo el nodo y dos subárboles (p. ej., Figura 1, luego Figura 2). Uno cuya raíz es la raíz del árbol original y otro cuya raíz es el hijo izquierdo si el nodo es un hijo izquierdo, o derecho si es un hijo derecho. Luego, se mueve el nodo a la raíz, lo que da como resultado el árbol de la Figura 3.

Se pide:

- Programar método en la clase `Arbol` `imprimirPreorden()` que imprima el árbol en preorden.
- Que programe un método en la clase `Arbol` `buscar(n)`, con `n` entero, que busque el número y retorne `True` si el elemento está en el árbol y `False` en caso contrario.

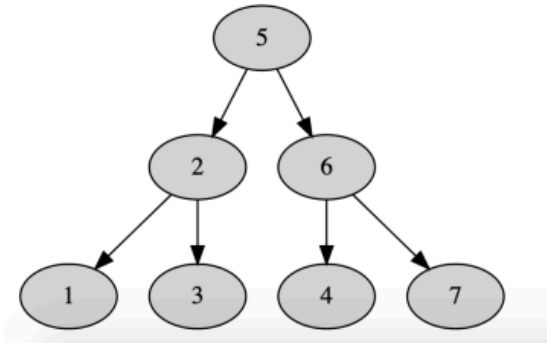
P3. Chequear si un árbol binario es de búsqueda

Los árboles ABB son una estructura de datos que cumplen las siguientes propiedades:

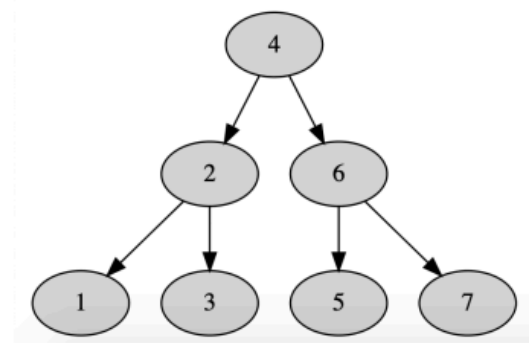
- Es un árbol binario, es decir, tiene un nodo raíz y a lo más dos subárboles en cada nivel.
- Todos los elementos pertenecientes al subárbol derecho, son mayores que la raíz.
- Todos los elementos pertenecientes al subárbol izquierdo, son menores que la raíz.

La definición del árbol contará con la siguiente estructura, similar a la que ya se ha visto en clases:

```
class Nodo :  
    def __init__ ( self , izq , info , der ) :  
        self.izq = izq  
        self.info = info  
        self.der = der  
  
class Arbol :  
    def __init__ ( self , raiz = None ) :  
        self.raiz = raiz
```



(a) Árbol que NO es de búsqueda binaria



(b) Árbol de búsqueda binaria

Programe una función `def checkABB(nodo)` que, dado un árbol binario, retorne `True` si es de búsqueda o `False` en caso contrario.